The `multitone` Package — Essay and User Manual

1. What It Is, and Why

`multitone` is a Python library for generating **multitone audio test signals** whose frequencies are **prime numbers**, whose per-frequency attenuation is carefully pre-computed, and whose phases are **optimized** to minimize the signal's **crest factor**. It is a faithful port of a MATLAB codebase (the docstrings consistently reference the original `.m` files: `MuTPrimes.m`, `FrqOctFraction.m`, `FrqLstVolAtt.m`, `PhiLstMuTNewMin.m`, `PhiLstMuTNewSig.m`).

The use case is audio-hardware and audio-software testing: a multitone signal exercises every part of an audio chain simultaneously — DACs, ADCs, amplifiers, room acoustics, loudspeakers, headphones — with a single, continuous stimulus. By choosing prime-number frequencies, the signal avoids harmonic relationships between tones, so intermodulation products are spread across the spectrum rather than stacking on the same bins. By minimizing the crest factor (the ratio of peak to RMS amplitude), the signal can run as loud as possible before clipping, maximizing the energy delivered to the device under test.

---

2. Theoretical Foundations

2.1 Prime-Based Frequency Selection

The frequency list is built in two stages:

1. **Octave subdivision** (`frq_oct_fraction`): Each octave band i (where the band spans 2^i Hz to 2^{i+1} Hz, from octave 4 = 16 Hz up to octave 18 = 262 144 Hz) is divided into *fraction* equal steps (3, 6, or 12 subdivisions). This yields a uniformly-spaced set of candidate frequencies within each octave.
2. **Prime mapping** (`mut_primes`): For each candidate frequency f, the largest prime $p \leq f$ is taken (via `sympy.primerange`). The result is deduplicated and filtered to the audible range of interest (typically 20 Hz – 48 kHz).

Why primes? If two tones at frequencies f_1 and f_2 are harmonically related (e.g., $f_2 = 2 \cdot f_1$), then their intermodulation products (sum and difference frequencies) fall back on the same harmonic grid, reinforcing each other and creating localized peaks. Primes are, by definition, not multiples of each other, so intermodulation products are distributed across the spectrum, producing a flatter overall frequency response — the hallmark of a good test signal.

2.2 Per-Frequency Attenuation

The `.mat` files in `data/` (`fr03pri384k.mat`, `fr06pri384k.mat`, `fr12pri384k.mat`) contain pre-computed attenuation curves (in dB) for each prime frequency. The finer the octave subdivision, the more frequencies, and the finer the frequency-resolution of the test signal. These attenuation values shape the overall spectral envelope — typically following an equal-energy-per-octave or pink-noise-like roll-off so that low-frequency tones (which carry less energy for the same amplitude) do not dominate.

2.3 Crest Factor and Its Minimization

The **crest factor** is defined as:

$$\text{CF} = 20 \cdot \log_{10}(\text{peak} / \text{RMS})$$

A high crest factor means the signal has sharp peaks relative to its average power. This is undesirable for test signals because:

- The average power is limited by the peak (to avoid clipping).

- The peaks themselves can cause intermodulation distortion in the device under test, even if the average power is modest.

The optimizer (`optimizer.py`) uses a **Monte Carlo** approach: it generates random phase vectors (uniform in [0, π]) and, for each, synthesizes the full waveform and measures the crest factor. Whenever a new minimum is found, the phase set and a short preview audio are saved. The optimizer can run for hundreds of thousands of iterations — typical best crest factors for dense multitone signals fall in the 8–10 dB range (versus 15–20 dB for random phases).

This is a well-known technique in the audio test-signal literature (e.g., the "Prime Tone" signals developed by Audio Precision and others). The theoretical insight is that with N tones, there are N degrees of freedom in the phase domain; by searching this space stochastically, one can find phase combinations that cause the peaks of individual cosines to destructively interfere with each other, flattening the envelope.

2.4 Synthesis and Quantization

The synthesis engine (`synthesize.py`) accumulates cosine terms:

$$y(t) = \Sigma_k \; 10\text{^}(a_k/20) \cdot \cos(2\pi \cdot f_k \cdot t + \varphi_k)$$

where a_k is the per-frequency attenuation (dB → linear), f_k the frequency, and φ_k the optimized phase. After accumulation:

1. **Peak normalization**: the signal is scaled so its peak equals full scale.
2. **Overall attenuation** (VolAtt, default −0.25 dB): a safety margin below full scale.
3. **Quantization**: round-to-nearest (RD) or truncate (NO) to the target bit depth (16, 24, or 32). For 24-bit, the lower 8 bits of an int32 container are zero-padded (×256), matching the MATLAB convention.

---

3. Architecture at a Glance

```
multitone/├──
 __init__.py          # Package declaration├──
 primes.py            # Frequency generation (octave subdivision → prime mapping)├──
 synthesize.py        # Waveform synthesis + crest factor computation├──
 optimizer.py         # Monte Carlo phase optimization (CLI: python -m
    ↪multitone.optimizer)├──
 producer.py          # Production signal generation from optimized phase (CLI:
    ↪python -m multitone.producer)├──
 data/│
    ├── fr03pri384k.mat  # Attenuation data for 3 subdivisions│
    ├── fr06pri384k.mat  # Attenuation data for 6 subdivisions│
    └── fr12pri384k.mat  # Attenuation data for 12 subdivisions└──
 requirements.txt     # numpy, scipy, soundfile, sympy
```

---

4. User Manual

4.1 Installation

```
pip install numpy scipy soundfile sympy
```

The package is importable directly from the `multitone` directory:

```
from multitone.primes import mut_primes, frq_oct_fraction
from multitone.synthesize import frq_lst_vol_att, crest_factor_dB
```

4.2 `primes.py` — Frequency List Generation

`frq_oct_fraction(oct0, oct1, fraction)` Generates linearly-subdivided frequencies within octave bands.

Parameters:

- `oct0` (int): Starting octave (4 → 16 Hz floor).
- `oct1` (int): Ending octave (18 → 262 144 Hz floor).
- `fraction` (int): Subdivisions per octave (3, 6, or 12).

Returns: `(frq_lst: ndarray, frq_len: int)`

Example:

```
from multitone.primes import frq_oct_fraction

# 3 subdivisions per octave, octaves –46
frq_lst, frq_len = frq_oct_fraction(4, 6, 3)
# → 10 frequencies: 16, 29, 43, 57, 85, 128, 171, 256, 341, 512 (approx.)
print(frq_len)  # 10
```

`mut_primes(fraction, frq_lim_low, frq_lim_high)` Generates the full prime-based frequency list with attenuation-ready output.

Parameters:

- `fraction` (int): Octave subdivisions (3, 6, or 12).
- `frq_lim_low` (float): Lower frequency limit (Hz).
- `frq_lim_high` (float): Upper frequency limit (Hz).

Returns: `(frq_pri_lim: ndarray, frq_pri_len: int)`

Example:

```
from multitone.primes import mut_primes

# 12 subdivisions, audible range
freqs, n = mut_primes(12, 20.0, 48000.0)
print(f"{n} prime frequencies from {freqs[0]} Hz to {freqs[-1]} Hz")
# → e.g., "273 prime frequencies from 23 Hz to 47929 Hz"
```

4.3 `synthesize.py` — Waveform Synthesis

`frq_lst_vol_att(FrqLst, PhiLst, VolLst, VolAtt, Rounded, Dur, Frs, Sab, NCh)` Synthesizes a multitone waveform.

Parameters:

- `FrqLst` (ndarray): Frequencies (Hz).
- `PhiLst` (ndarray): Phases (radians).
- `VolLst` (ndarray): Per-frequency attenuation (dB).
- `VolAtt` (float): Overall peak attenuation (dB). Negative = back-off.
- `Rounded` (str): `'RD'` for round-to-nearest, `'NO'` for truncate.
- `Dur` (int): Duration (seconds).
- `Frs` (int): Sampling rate (44100, 48000, 88200, 96000, 176400, or 192000).
- `Sab` (int): Bit depth (16, 24, or 32).
- `NCh` (int): Channels (1 or 2).

Returns: `ycd` (ndarray): Quantized audio, shape `(samples, NCh)`.

Example:

```
import numpy as np
from multitone.synthesize import frq_lst_vol_att

# 5 prime frequencies, random phases
freqs = np.array([101, 103, 107, 109, 113])
phases = np.random.rand(5) * np.pi
attenuations = np.zeros(5)  # no per-frequency attenuation

ycd = frq_lst_vol_att(
    FrqLst=freqs,
    PhiLst=phases,
    VolLst=attenuations,
    VolAtt=-3.0,        # 3 dB peak back-off
    Rounded="RD",
    Dur=1,               # 1 second
    Frs=48000,
    Sab=24,              # 24-bit
    NCh=2,               # stereo
)
print(ycd.shape)  # (48000, 2)
```

`crest_factor_dB(ycd, Sab)` Computes the crest factor in dB per channel.

Example:

```
from multitone.synthesize import crest_factor_dB

cf = crest_factor_dB(ycd, 24)
print(f"Crest factor: {cf[0]:.2f} dB (L), {cf[1]:.2f} dB (R)")
# → e.g., "Crest factor: 9.12 dB (L), 9.12 dB (R)"
```

4.4 `optimizer.py` — Monte Carlo Phase Optimization

Runs a Monte Carlo search for the phase combination that minimizes crest factor.

CLI usage:

```
# Default: Fraction=12, 96kHz, 24-bit, mono, 100k iterations
python -m multitone.optimizer

# Custom: Fraction=3, 48kHz, stereo, 50k iterations, seed=42
python -m multitone.optimizer \
    --fraction 3 --fsam 48000 --nch 2 \
    --maxiter 50000 --seed 42 --output-dir ./results

# Resume from iteration 50001
python -m multitone.optimizer --last-best-iter 50001

# Quick test: 10 iterations, 1-second preview
python -m multitone.optimizer --maxiter 10 --dur 1
```

Full CLI options:

Flag	Type	Default	Description

‘–fraction‘	int	12	Octave subdivisions (3, 6, 12)
‘–fsam‘	int	96000	Sampling rate (44100...192000)
‘–sbit‘	int	24	Bit depth (16, 24, 32)
‘–ilowfreq‘	int	4	Frequency-array slice start (1-indexed)
‘–nch‘	int	1	Channels (1 or 2)
‘–attvol‘	float	-0.25	Peak attenuation (dB)
‘–dur‘	int	1	Preview duration (seconds)
‘–maxiter‘	int	100000	Maximum iterations
‘–last-best-iter‘	int	1	Resume from this iteration
‘–rd‘	str	RD	Rounding (RD or NO)
‘–ext‘	str	.flac	Audio extension
‘–seed‘	int	None	Random seed
‘–output-dir‘	str	.	Output directory

Programmatic usage:

```
from multitone.optimizer import optimize

optimize(
    fraction=6,
    fsam=48000,
    sbit=24,
    ilowfreq=4,
    nch=1,
    attvol=-0.25,
    dur=1,
    maxiter=50000,
    seed=42,
    output_dir="./results",
)
```

Output: For each improvement, the optimizer saves:

- A `.mat` phase file (compatible with MATLAB) named `PhiLstMuTNewMin[...].mat` containing the best phases, frequencies, and crest factor.
- A preview audio file (`.flac` or `.wav`) named `PrimeToneNewPhi[...].flac`.

4.5 `producer.py` — Production Signal Generation

Takes an optimized phase file and generates a long-duration audio signal.

CLI usage:

```
# 30-minute stereo FLAC from a phase file
```

```
python -m multitone.producer \
    "PhiLstMuTNewMin[Nch 1][Frac 03][LFL 043][048000-24][00020-21600][Nfr
    ↪027][060159]-[ 8.4655].mat" \
    --dur 1800 --nch 2

# Quick 1-second mono WAV for verification
python -m multitone.producer phase_file.mat --dur 1 --nch 1 --ext .wav
```

Programmatic usage:

```
from multitone.producer import produce

produce(
    phase_mat_path="PhiLstMuTNewMin[...].mat",
    dur=1800,           # 30 minutes
    nch=2,              # stereo
    attvol=-0.25,
    rd="RD",
    ext=".flac",
)
```

---

5. End-to-End Workflow

1. **Generate frequencies** (optional — the optimizer uses pre-computed data, but you can inspect):

```
from multitone.primes import mut_primes
freqs, n = mut_primes(12, 20.0, 48000.0)
print(f"{n} prime tones")
```

2. **Optimize phases** (this is the heavy step):

```
python -m multitone.optimizer --fraction 12 --fsam 96000 --maxiter 100000 --seed 42
```

3. **Generate the production signal** from the best phase file:

```
python -m multitone.producer "PhiLstMuTNewMin[...].mat" --dur 1800 --nch 2
```

4. **Verify** the result:

```
import numpy as np
import soundfile as sf
from multitone.synthesize import crest_factor_dB

ycd, sr = sf.read("PrimeToneNewPhi[...].flac")
cf = crest_factor_dB(ycd, 24)
print(f"SR: {sr}, Samples: {ycd.shape}, Crest factor: {cf[0]:.4f} dB")
```

---

6. Design Decisions and MATLAB Parity

The package was built as a **1:1 port** of MATLAB scripts. Key parity points:

- **Cosine (not sine)** basis functions — matches the MATLAB `cos` calls.
- **24-bit encoding** uses int32 containers with lower 8 bits zero-padded (×256), matching MATLAB's convention for 24-bit audio.

- **Peak normalization** is applied before overall attenuation, exactly as in the MATLAB code.
- **Phase range** is [0, π] (not [0, 2π]), matching rand(N,1)*pi.
- **MAT file I/O** uses `scipy.io` for full interoperability — phase files from the Python optimizer can be loaded directly in MATLAB and vice versa.

The one notable **improvement** over the MATLAB original: the attenuation-frequency matching in `producer.py` uses a dictionary lookup (`att_lookup`) instead of relying on array-length coincidence, which was a latent bug in the MATLAB code where sliced frequency lists and full attenuation lists were passed to `FrqLstVolAtt` with potentially mismatched lengths.